



preparatic 31

TEMA 067

ENTORNO DE DESARROLLO JAVA

Versión	31.1
Fecha de actualización	19/02/2026

ÍNDICE

1	Introducción.....	2
2	Plataforma	2
2.1	Estructura	2
2.2	Ediciones	3
3	Java Enterprise (Jakarta EE)	4
3.1	Especificaciones	4
3.2	Implementaciones (Servidores)	5
4	Ecosistema	6
4.1	Librerías.....	6
4.2	Herramientas	7
4.3	Frameworks	7
4.4	Spring	7
4.5	Lenguajes sobre la JVM.....	8
5	Glosario	8
1.1.	Bytecode:.....	8
1.2.	Recolección de basura:	8
1.3.	Orientación a objetos:	9
6	Referencias	9

1 Introducción

Java es un lenguaje de programación y una plataforma de construcción de aplicaciones que fue creada por Sun Microsystems en 1995, posteriormente adquirida por Oracle Corp en 2010.

Es un lenguaje de programación de alto nivel cuya principal característica es el denominado **“Write Once, Run Anywhere”** que lo convierte en un lenguaje multi-plataforma. Otras características importantes es la orientación a objetos y la gestión de memoria automática (garbage collector), en sus últimas versiones incluye opciones de programación funcional o concurrente.

Para mantener la relevancia desde hace tanto tiempo Java ha evolucionado mucho con el tiempo, incluyendo nuevas funcionalidades tanto en las librerías incluidas en el core, como el propio lenguaje.

Aunque Oracle mantiene la marca comercial es un desarrollo 100% open-source y existe un organismo formal que aprueba que se incluye en cada versión JDK (Java Development Kit) denominado JCP (Java Community Process) compuesto por empresas y desarrolladores individuales que proponen cambios o mejoras.

2 Plataforma

Podríamos denominar la plataforma Java como un entorno de software compuesto por un lenguaje de programación de alto nivel, un conjunto de bibliotecas estandarizadas y un motor de ejecución virtualizado.

2.1 Estructura

Podemos estructurar la plataforma en las siguientes partes:

- **Java Programming Language (El Lenguaje):** Como ya hemos indicado es un lenguaje orientado a objetos, fuertemente tipado y concurrente. Se caracteriza por compilar el código fuente (.java) en un formato intermedio independiente de la plataforma llamado Bytecode (.class).
- **Java Virtual Machine (JVM):** Es el corazón de la plataforma. Es un intérprete de software que ejecuta el Bytecode. La JVM actúa como una capa de abstracción entre el programa y el sistema operativo, encargándose de la gestión de memoria (Garbage Collection) y la seguridad en tiempo de ejecución. Además controla la carga de las clases y el acceso a las mismas, desde el Java 9 es un sistema de módulos que transformó la arquitectura monolítica del JDK en una estructura de componentes interconectados.
- **Java API (Application Programming Interface):** Una colección extensa de componentes de software pre-escritos (clases e interfaces) que proporcionan capacidades listas para usar, desde manejo de redes y archivos hasta criptografía y estructuras de datos complejas.

2.2 Ediciones

Dependiendo del entorno de aplicación, la plataforma se distribuye formalmente en diferentes ediciones:

- **Java SE** (Standard Edition): La base de la plataforma, utilizada para aplicaciones de escritorio y servidores de propósito general.
- **Java EE / Jakarta EE** (Enterprise Edition): Una extensión de SE diseñada para aplicaciones empresariales a gran escala, distribuidas y multicapa.
- **Java ME** (Micro Edition): Una versión reducida para dispositivos con recursos limitados (sensores, microcontroladores).

No hay que confundir el **JDK** (Kit de Desarrollo) con el **JRE** (Entorno de Ejecución). El JDK es lo que usa el programador para crear el software, mientras que el JRE (que contiene la JVM) es lo mínimo necesario para que un usuario final pueda ejecutarlo.

OpenJDK (Open Java Development Kit) es la **implementación de referencia de código abierto** de la plataforma Java SE, bajo la licencia pública **GNU GPL**. Es el proyecto donde ocurre el desarrollo real del lenguaje: cada nueva función, corrección de errores o mejora de rendimiento de Java se escribe y se prueba primero en OpenJDK.

Aunque Oracle lidera el proyecto, OpenJDK es un esfuerzo **colaborativo** global donde participan grandes empresas como Microsoft, Amazon, Red Hat, Google e IBM, quienes contribuyen con código para asegurar que Java funcione de forma óptima en sus propias entornos cloud y sistemas. En la práctica, casi todas las versiones comerciales de Java que usamos hoy (como Oracle JDK, Amazon Corretto o Microsoft Build of OpenJDK) son simplemente "sabores" o distribuciones construidas a partir del código fuente gratuito de OpenJDK.

La versión de Oracle JDK ha perdido seguidores al priorizar las versiones con soporte comercial de pago.

Por contra, **GraalVM**, también de Oracle, es considerada una de las evoluciones más importantes del ecosistema Java en los últimos años. Puede funcionar como un JVM de alto rendimiento con un compilador JIT (Just-In-Time) o como un compilador de imágenes nativas AOT (Ahead-Of-Time) que transforma el bytecode de Java en un **binario ejecutable nativo** e independiente, eliminando la necesidad de una JVM para lograr un arranque instantáneo y un consumo de memoria mínimo.

Como referencia de la frecuencia de actualización del *JDK de Oracle* se muestra un cuadro de publicaciones y su soporte gratuito. Solo se incluyen las versiones LTS (Long-Term Support):

Versión	Lanzamiento	Fin actualizaciones	Novedad principal
8	Marzo 2014	Abril 2019 *	Programación funcional: lambdas y API streams
11	Septiembre 2018	Abril 2019 *	Cliente HTTP nativo con soporte HTTP/2
17	Septiembre 2021	Septiembre 2024	Sealed classes y migración javax.* a jakarta.* en JakartaEE 9
21	Septiembre 2023	Septiembre 2026	Virtual Threads (Project Loom)
25	Septiembre 2025	Septiembre 2029 **	Optimización masiva de la memoria con Compact Object Headers

* Oracle cambió a licencia OTNL (Oracle Technology Network License) que solo permite usos personales, no comerciales

** Fecha estimada

3 Java Enterprise (Jakarta EE)

Jakarta EE (anteriormente conocido como Java EE o J2EE) es el estándar de la industria para el desarrollo de aplicaciones **empresariales** robustas, escalables y seguras en Java.

Es un conjunto de especificaciones (reglas de juego) que definen cómo deben construirse los componentes críticos de un servidor: desde la conexión a bases de datos hasta la creación de APIs REST y la seguridad de identidades.

En 2017, Oracle donó Java EE a la Eclipse Foundation. Debido a restricciones de marca con la palabra "Java", el proyecto fue renombrado a Jakarta EE. El cambio más significativo ocurrió en Jakarta EE 9, donde todos los paquetes pasaron de llamarse javax.* a jakarta.*.

3.1 Especificaciones

No hay que confundir implementación (los servidores como WildFly o Glassfish) con la especificación (las "reglas": JPA, CDI, JAX-RS, ...). A continuación nombraremos alguna de las más importantes:

- **Jakarta RESTful Web Services (JAX-RS).** Usada para crear APIs REST. En lugar de configurar rutas manualmente, usas anotaciones sobre clases de Java simples (POJOs).
- **Contexts and Dependency Injection (CDI).** Se encarga de gestionar el ciclo de vida de tus objetos y de "inyectarlos" donde se necesiten.

- **Jakarta Persistence (JPA).** Estándar para interactuar con bases de datos sin escribir SQL manualmente
- **Jakarta Bean Validation.** Sirve para validar datos mediante anotaciones simples, extremadamente útil para asegurar que la información que llega desde un formulario o una API es correcta.

Cuadro resumen de las últimas versiones:

Versión	Año	Novedad Principal / Hito
Java EE 8	2017	Introdujo JSON-B (binding de objetos a JSON) y mejoras en CDI 2.0 . Fue la última versión antes de la donación a Eclipse.
Jakarta EE 8	2019	Es funcionalmente idéntica a Java EE 8. Su gran novedad fue el cambio de gobernanza ; el código ahora es Open Source.
Jakarta EE 9	2020	El " Big Bang " del Namespace . Se cambió <code>javax.*</code> por <code>jakarta.*</code> en todo el código. Rompió la retrocompatibilidad para siempre.
Jakarta EE 10	2022	Introdujo el Core Profile para microservicios. Eliminó tecnologías obsoletas y se optimizó para Java 17 y ejecución en la nube.
Jakarta EE 11	2024+	Alineación total con Java 21 . Soporte nativo para Virtual Threads (hilos virtuales) y mejoras masivas en seguridad y rendimiento.

Una aplicación construida siguiendo estas especificaciones debería poder ser desplegada en cualquier servidor que las implemente, evitando el “vendor lock-in”.

3.2 Implementaciones (Servidores)

Los **perfiles** en Jakarta EE nacieron para resolver un problema histórico, la inclusión de todas especificaciones en un servidor completo que obligaba a tener muchos servicios que no se usaban en la mayoría de aplicaciones.

En la versión 10 de Jakarta EE existen tres:

- **Web Profile:** Una versión ligera diseñada para aplicaciones web modernas (incluye Servlets, JPA, CDI y Jakarta REST).
- **Full Platform:** Incluye absolutamente todo, desde servicios de mensajería legacy hasta conectores para sistemas industriales antiguos.
- **Core Profile:** El más reciente (introducido en Jakarta EE 10) y ligero, diseñado específicamente para Microservicios. Incluye solo lo mínimo necesario para APIs REST y ejecución en contenedores. No incluye soporte JSF, solo Jakarta REST (JAX-RS), CDI (Inyección), JSON-P y JSON-B (Manejo de JSON).

Existen muchos fabricantes que implementan estas versiones.

Open source:

- **WildFly:** (Red Hat) El más popular por su velocidad de arranque y modularidad.
- **GlassFish:** La implementación de referencia mantenida por la Eclipse Foundation.
- **Apache TomEE:** Es Apache Tomcat web profile, añadiendo CDI, JPA y REST.
- **Open Liberty:** La versión abierta del servidor de IBM.
- **Helidon MicroProfile** (Oracle) orientado a ejecución de microservicios

Comerciales:

- **Red Hat JBoss Enterprise Application Platform (JBoss EAP):** La versión profesional y estable de WildFly (Red Hat/IBM).
- **Oracle WebLogic:** Enfocado en integraciones masivas de Oracle.
- **IBM WebSphere Liberty:** Muy ligero y diseñado para entornos híbridos.

Es importante destacar que **Apache Tomcat**, se creó como Servlet Container y **no es un perfil web** completo al carecer de alguna de las especificaciones (CDI, JPA, JAX-RS y otras). Igualmente *Jetty* o *Undertow* aunque son más modernos encajan en la misma definición. Existen librerías que se pueden añadir a una aplicación para incluir estas opciones pero no están integradas en el servidor.

4 Ecosistema

Ya hemos hablado las partes principales del entorno de desarrollo Java, pero existen multitud de herramientas y librerías que podemos denominar como ecosistema de desarrollo.

4.1 Librerías

Son pequeños paquetes de código que solucionan problemas específicos, con el paso del tiempo muchos tienen una equivalencia en el propio JRE:

- Log4j/SLF4J/Logback, para el registro de eventos (logs).

- Hibernate para gestionar la persistencia en BBDD relacionales, es una implementación de JPA.
- JUnit/Mockito, para testing
- Jackson, estándar de facto para trabajar con JSON
- Apache Commons / Google Guava, son ayudantes con infinidad de utilidades generales que el estándar no trae de forma tan sencilla.

4.2 Herramientas

Dada la longevidad de la tecnología Java existen multitud de herramientas que automatizan el ciclo de vida del desarrollo (construcción, pruebas y despliegue), permitiendo que el programador se abstraiga de la complejidad de la infraestructura para enfocarse en la lógica de negocio.

A continuación, nombramos algunas de las más importantes aunque existen muchísimas:

- **Maven/Gradle.** Son herramientas de construcción y gestores de dependencias, ayudan en todo el ciclo de vida del desarrollo
- **SonarQube:** Para realizar una inspección continua que analiza automáticamente el código fuente para detectar errores y vulnerabilidades de seguridad.
- **JMeter:** Para medir el rendimiento y realizar pruebas de carga sobre aplicaciones.
- **Jenkins:** Como herramienta de flujo de Integración y Despliegue Continuo (CI/CD), encargándose de compilar, probar y desplegar el código automáticamente cada vez que se realiza un cambio
- **IDEs** (Integrated Development Environment) para Java como: IntelliJ IDEA, Eclipse o VSCode.

4.3 Frameworks

Un framework es mucho más que una simple colección de librerías. La mejor forma de definirlo es como un esqueleto de software pre-construido que proporciona una estructura y una metodología de trabajo específica.

Ejemplos relevantes:

- Podemos considerar JakartaEE un framework para aplicaciones empresariales
- Spring / SpringBoot (que detallaremos a continuación)
- Quarkus, diseñado para optimizar el consumo de memoria y la velocidad de arranque enfocado a entornos de microservicios y Serverless.

4.4 Spring

Es uno de los frameworks más populares para la construcción de aplicaciones empresariales, más que JavaEE o JakartaEE, sobre todo porque cuando nació simplificaba en gran medida la construcción de estos sistemas en comparación a JavaEE y no requería

de un servidor completo, lo que, unido a la popularidad de Tomcat, lo convirtió en el método principal para el desarrollo de aplicaciones.

Construido en origen para proporcionar herramientas para la Inyección de Dependencias (DI) y la Inversión de Control (IoC), con el paso del tiempo y con la complejidad de crear un proyecto inicial se creó **Spring Boot**, una herramienta construida encima de Spring Framework como acelerador, aplicando el concepto de "Convención sobre Configuración".

Los elementos más habituales que podemos añadir, denominados *starters*, son:

- Desarrollo Web y servicios REST: *spring-boot-starter-web* o *spring-boot-starter-webflux* y *spring-boot-starter-thymeleaf*
- Persistencia y BBDD: *spring-boot-starter-data-jpa* o *spring-boot-starter-data-mongodb*
- Seguridad: *spring-boot-starter-security*
- Mensajería: *spring-boot-starter-amqp* y *spring-boot-starter-kafka*

4.5 Lenguajes sobre la JVM

Gracias a que la JVM no ejecuta código fuente directamente, sino bytecode (un lenguaje intermedio), cualquier lenguaje que pueda compilarse a ese formato puede correr sobre ella.

- **Kotlin**. Creado por JetBrains para ser más conciso y seguro. Es el lenguaje oficial para el desarrollo de **Android**.
- **Scala**. Primera implementación de programación funcional, muy orientado al tratamiento masivo de datos. Está detrás de Apache Spark, lo que lo convierte en el estándar en el mundo del Big Data.
- **Groovy**. Es un lenguaje "relajado" que se parece mucho a Java pero con una sintaxis mucho más flexible (similar a Python o Ruby). Es el lenguaje que utiliza Gradle.
- **Clojure**. Es un dialecto de Lisp. Es radicalmente diferente a Java porque es un lenguaje funcional basado en listas y paréntesis.

5 Glosario

1.1. Bytecode:

Bytecode se la llama en java al compilado que se produce con el código Java. Este compilado esta formado de instrucciones muy simples, las cuales son fácilmente traducibles a intrucciones nativas. Por tanto este código es muy muy rápido, casi nativo en la ejecución y mantiene la propiedad de que se puede usar en muchas máquinas diferentes.

1.2. Recolección de basura:

Es un método de gestión de memoria en programación. Se basa en que se sigue la ejecución del programa registrando cada vez que se adquiere memoria ram. Cuando

lógicamente es imposible que esa misma memoria se vuelva a usar, será detectado y liberada. Esto libera al programador de la tarea de gestión de memoria aumentando la productividad a costa de un ejecutable más lento.

1.3. Orientación a objetos:

Es un paradigma de programación basado en el concepto de objeto. Esto es que la información y las instrucciones para procesarla han de ir de la mano dando lugar a fenómenos, que incrementan la productividad de los programadores y la mantenibilidad del código, como la modularidad, el encapsulamiento, abstracción, reusabilidad.

6 Referencias

Básicas:

- Lenguaje Java (wikipedia): [https://en.wikipedia.org/wiki/Java_\(programming_language\)](https://en.wikipedia.org/wiki/Java_(programming_language))
- Documentación Java (de Oracle): <https://dev.java/> versión 25: <https://docs.oracle.com/en/java/javase/25/>
- Especificaciones Jakarta EE: <https://jakarta.ee/specifications/>
- Versiones de Oracle Java: <https://www.java.com/releases/>
- Documentación de Spring framework: <https://docs.spring.io/spring-framework/reference/>

Adicionales:

- OpenJDK 25: <https://jdk.java.net/25/>
- Soporte Java de Oracle: <https://www.oracle.com/nz/java/technologies/java-se-support-roadmap.html>
- Apache TomEE: <https://tomee.apache.org/>
- Apache Maven: <https://maven.apache.org/>
- Spring Boot: <https://spring.io/projects/spring-boot>
- Versiones de Java (wikipedia): https://en.wikipedia.org/wiki/Java_version_history
- Kotlin: <https://kotlinlang.org/>
- Quarkus: <https://es.quarkus.io/>
- Amazon JVM Coretto: <https://aws.amazon.com/es/corretto/>

